

• Quais são as entidades (classes) que devem ser criadas?

Para modelar este cenário utilizando os conceitos de Orientação a Objetos e DDD (*Domain-Driven Design*), devemos criar duas entidades principais e duas estruturas de suporte:

1. **Veiculo (Entidade)**: Classe responsável por identificar o meio de transporte que está utilizando o estacionamento (armazenando dados essenciais como a placa).
2. **Ticket (Entidade)**: A classe central do sistema (raiz de agregação). Ela controla todo o ciclo de vida da permanência do veículo, registrando o momento exato da entrada, o momento da saída, o valor total devido e o status atual do pagamento.
3. **TipoVeiculo (Enum/Estrutura)**: Um enumerador para limitar e padronizar as opções de porte exigidas pelo cenário: **Carro**, **Moto** e **Caminhao**.
4. **FormaPagamento (Enum/Estrutura)**: Um enumerador para restringir as opções de acerto financeiro: **Dinheiro** e **Cartao**.
5. **CalculadoraTarifa (Classe de Serviço)**: Uma classe utilitária e sem estado (*stateless*) focada puramente em isolar o cálculo matemático dos preços.

• Quais dessas entidades possuem ações (métodos) e quais são eles?

Seguindo as boas práticas de desenvolvimento para evitar modelos anêmicos (classes que só guardam dados, mas não têm comportamento), as ações foram distribuídas da seguinte forma:

1. **Na classe Ticket**:
 - **RegistrarSaida(DateTime horarioSaida, decimal valor)**: Altera os estados temporais do ticket (registra o fim da permanência) e fixa o valor monetário final que o cliente deve pagar.
 - **ProcessarPagamento(FormaPagamento forma)**: Modifica o status interno do ticket para "Pago" e vincula de forma definitiva qual foi o método financeiro utilizado.
2. **Na classe CalculadoraTarifa**:
 - **Calcular(TipoVeiculo tipo, TimeSpan tempoPermanencia)**: Método estático ou de serviço que recebe o tipo do automóvel e a duração da estadia, processa a regra matemática do negócio e retorna o valor monetário exato a ser cobrado.

• Há necessidade de criação de interface? Se sim, descreva os comportamentos que devem ser criados.

No cenário base básico não há uma obrigatoriedade estrita, porém, se pensarmos em boas práticas de arquitetura, testabilidade e no princípio aberto/fechado do SOLID, a criação de duas interfaces é extremamente recomendada:

1. **ICalculadoraTarifa**: Isola as regras de cobrança. Se no futuro o estacionamento decidir cobrar por minutos, adotar tarifas noturnas, ou dar descontos para convênios, bastará criar uma nova classe que implemente esta interface, sem mexer na estrutura do **Ticket**.
 - **Comportamento**: `decimal Calcular(TipoVeiculo tipo, TimeSpan tempoPermanencia);`

2. **IProcessadorPagamento:** Abstrai o ecossistema financeiro. O sistema de estacionamento não precisa saber como a máquina de cartão funciona por dentro. Essa interface encapsula a comunicação com gateways ou maquininhas.
 - *Comportamento:* `bool AutorizarTransacao(decimal valor);`

• Quais são os relacionamentos entre as classes e entidades criadas?

Os relacionamentos do sistema definem como as peças conversam entre si na memória:

1. **Associação Unidirecional (1 para 1) entre Ticket e Veiculo:** Um `Ticket` conhece exatamente um `Veiculo`. O veículo até possui existência independente, mas o ticket não faz sentido e não pode existir se não estiver vinculado a um carro, moto ou caminhão.
2. **Composição de Propriedades:** O `Veiculo` possui/contém um `TipoVeiculo` mapeado nele. Da mesma forma, o `Ticket` possui/contém uma `FormaPagamento` associada quando é liquidado.
3. **Dependência Temporária:** A classe `Ticket` (ou o fluxo que gerencia o ticket) possui uma dependência em relação à `CalculadoraTarifa` no momento do fechamento da conta, pois necessita do resultado do cálculo dela para preencher seu próprio atributo de valor.

• Onde o desenvolvedor deve tratar exceções em caso de críticas no processo?

O tratamento de exceções e validações de críticas deve ser tratado em duas camadas distintas e complementares:

1. **Na Camada de Domínio (Dentro das próprias Entidades - Estratégia *Fail-Fast*):** As entidades devem se autoprotger contra dados corrompidos ou impossíveis de acontecer na vida real. O desenvolvedor deve lançar exceções de argumento ou de operação (`ArgumentException` ou `InvalidOperationException`) diretamente nos métodos e construtores se:
 - Alguém tentar dar entrada em um veículo com a placa em branco.
 - O horário de saída for menor ou igual ao horário de entrada (tempo negativo).
 - Alguém tentar efetuar o pagamento de um ticket que já consta no banco de dados como "Pago".
2. **Na Camada de Aplicação / Controladores (Interfaces, APIs ou Telas):** É aqui que o desenvolvedor abre blocos `try-catch`. Essa camada captura os erros brutos gerados pela camada de domínio e realiza duas tarefas fundamentais: grava um log técnico detalhado em arquivos ou servidores de monitoramento para auditoria dos desenvolvedores, e traduz o erro em uma mensagem amigável e limpa na tela para o operador de caixa (ex: *"Operação negada: Este veículo já efetuou o pagamento anteriormente."*).